L19 - MoE 混合专家模型

> **"术业有专攻"**

本节目标

1. 理解 Dense 模型与 Sparse 模型的区别
2. 掌握 MoE 的核心原理：Router + 多个专家
3. 理解 Top-K Routing 和负载均衡 Loss
4. 了解 MiniMind-3-MoE 的具体配置与实现
5. 分析 MoE 的优劣与训练挑战

前置知识

- L05 FFN 的结构与作用
- L06 完整 Transformer Block 的构成
- Softmax 函数
- 基本的矩阵运算

一、Dense 模型 vs Sparse 模型

1.1 Dense 模型

传统 Transformer 是 **Dense（稠密）** 模型：每个输入 token 都要经过**所有**参数的计算。

```
输入 token → Attention (所有参数) → FFN (所有参数) → 输出
```

参数量 = 激活参数量。如果模型有 1B 参数，每个 token 都要用到全部 1B 参数。

1.2 Sparse 模型

MoE 是一种 **Sparse（稀疏）** 模型：模型有很多参数，但每个 token 只激活其中一部分。

```
输入 token → Attention (所有参数) → Router → 选择部分 FFN 专家 → 输出
```

总参数量 >> 激活参数量。比如 MiniMind-3-MoE： - 总参数量：198M - 激活参数量：64M（每个 token 只用到 64M 参数）

1.3 为什么要 Sparse?

核心动机：我们希望模型有更大的"知识容量"（更多参数），但不想让推理速度变慢（更多计算量）。

```
Dense 1B:  参数=1B,  每 token 计算量 ∝ 1B
MoE 4B:   参数=4B,  每 token 计算量 ∝ 1B  ← 更大容量，同等速度
```

二、MoE 的核心原理

2.1 整体架构

MoE 的改造只发生在 **FFN 层**，Attention 层不变：

```
标准 Transformer Block:
    x → LayerNorm → Attention → + → LayerNorm → FFN → + → 输出

MoE Transformer Block:
    x → LayerNorm → Attention → + → LayerNorm → MoE(FFN₁, FFN₂, ..., FFNₙ) → + → 输出
```

每个"专家"就是一个独立的 FFN 网络，结构和标准 FFN 完全相同（都是 SwiGLU 或类似结构），只是参数不同。

2.2 Router（路由器）

Router 是 MoE 的核心组件，负责决定每个 token 应该送给哪个专家处理。

```
class Router(nn.Module):
    def __init__(self, hidden_dim, num_experts):
        super().__init__()
        self.gate = nn.Linear(hidden_dim, num_experts, bias=False)

    def forward(self, x):
        # x: [batch_size, seq_len, hidden_dim]
        logits = self.gate(x)  # [batch_size, seq_len, num_experts]
```

```
        probs = F.softmax(logits, dim=-1)
        return probs
```

Router 本质上就是一个线性层 + Softmax，输出每个 token 对每个专家的"偏好分数"。

2.3 Top-K Routing

得到 Router 的输出后，选择分数最高的 K 个专家来处理该 token：

```
def top_k_routing(router_probs, k=1):
    # router_probs: [batch, seq_len, num_experts]
    topk_values, topk_indices = torch.topk(router_probs, k, dim=-1)
    # 归一化选中专家的权重
    topk_weights = topk_values / topk_values.sum(dim=-1, keepdim=True)
    return topk_weights, topk_indices
```

Top-1 Routing（MiniMind 使用的方式）：每个 token 只送给**1 个**专家。 - 优点：计算量最小 - 缺点：每个 token 的信息只经过一个专家，表达能力受限

Top-2 Routing（更常见的选择）：每个 token 送给 **2 个**专家，输出是两个专家的加权和。 - 优点：表达能力更强 - 缺点：计算量翻倍

2.4 MoE 前向传播

完整的 MoE 层前向传播：

```
class MoELayer(nn.Module):
    def __init__(self, hidden_dim, ffn_dim, num_experts, top_k):
        super().__init__()
        self.experts = nn.ModuleList([
            FFN(hidden_dim, ffn_dim) for _ in range(num_experts)
        ])
        self.router = Router(hidden_dim, num_experts)
        self.top_k = top_k

    def forward(self, x):
        # 1. Router 计算路由权重
        router_probs = self.router(x)  # [B, T, num_experts]

        # 2. Top-K 选择
        weights, indices = torch.topk(router_probs, self.top_k, dim=-1)
        weights = weights / weights.sum(dim=-1, keepdim=True)

        # 3. 分发给专家并加权汇总
        output = torch.zeros_like(x)
        for i, expert in enumerate(self.experts):
            # 找到选择了专家 i 的 token
            mask = (indices == i).any(dim=-1)  # [B, T]
            if mask.any():
```

```
                expert_input = x[mask]
                expert_output = expert(expert_input)
                # 加权累加
                weight = weights[indices == i]
                output[mask] += weight.unsqueeze(-1) * expert_output

        return output
```

三、负载均衡 Loss

3.1 专家塌缩问题

MoE 训练中最大的隐患是**专家塌缩（Expert Collapse）**：所有 token 都涌向同一个专家，其他专家得不到训练。

```
理想情况:                    塌缩情况:
Expert 1: 25% tokens        Expert 1: 95% tokens ← 过载
Expert 2: 25% tokens        Expert 2: 2% tokens
Expert 3: 25% tokens        Expert 3: 2% tokens
Expert 4: 25% tokens        Expert 4: 1% tokens  ← 几乎废弃
```

为什么会塌缩？ 因为正反馈循环——某个专家一开始碰巧表现好 → 更多 token 被路由到它 → 它得到更多训练 → 表现更好 → 更多 token 路由过来...

3.2 负载均衡 Loss

为了防止塌缩，需要添加一个辅助损失来鼓励均匀分配：

$$\mathcal{L}_{balance} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

其中： - N：专家数量 - f_i：实际被路由到专家 i 的 token 比例（频率） - P_i：Router 对专家 i 的平均概率 - α：平衡系数（通常 0.01）

直觉理解：如果某个专家同时有高频率（f_i 大）和高概率（P_i 大），loss 就会大，梯度会推动 Router 把一些 token 分给其他专家。

```
def load_balance_loss(router_probs, expert_indices, num_experts):
    # router_probs: [B*T, num_experts] - Router 输出的概率
    # expert_indices: [B*T] - 实际选择的专家索引

    # f_i: 每个专家被选中的频率
    freq = torch.zeros(num_experts, device=router_probs.device)
    for i in range(num_experts):
        freq[i] = (expert_indices == i).float().mean()
```

```
    # P_i: 每个专家的平均路由概率
    avg_prob = router_probs.mean(dim=0)  # [num_experts]

    # 负载均衡 loss
    loss = num_experts * (freq * avg_prob).sum()
    return loss
```

3.3 总训练损失

MoE 模型的训练损失是语言模型损失加上负载均衡损失：

$\mathcal{L}_{total} = \mathcal{L}_{LM} + \alpha \cdot \mathcal{L}_{balance}$

四、MiniMind-3-MoE 详细配置

4.1 模型参数

配置项	值
专家数量	4
Top-K	1（每个 token 选 1 个专家）
总参数量	198M
激活参数量	64M（≈ MiniMind 基础版）
隐藏维度	与基础版相同
FFN 维度	与基础版相同
层数	与基础版相同

4.2 参数量计算

为什么总参数是 198M 而激活参数是 64M？

```
基础版 (Dense):
  Embedding + Attention + FFN = 64M

MoE 版:
  Embedding + Attention = X  (不变)
  FFN 部分: 每层 1 个 FFN → 每层 4 个 FFN
  FFN 参数 = 4 × 原来的 FFN 参数
  Router 参数 ≈ 很小（hidden_dim × num_experts per layer）
```

```
总参数 ≈ X + 4 × FFN_params ≈ 198M
激活参数 ≈ X + 1 × FFN_params ≈ 64M（Top-1 只激活 1 个专家）
```

4.3 与 Dense 版的效果对比

根据 MiniMind 项目的实验： - **相同激活参数量（64M）**：MoE 版本通常比 Dense 版本效果更好，因为有更大的参数容量 - **相同总参数量**：Dense 版本通常更好，因为每个 token 都用到了所有参数 - **训练速度**：MoE 在原生 PyTorch 上训练比 Dense 慢（kernel 调度开销）

五、MiniMind MoE 源码解读

5.1 MoE 层定义

MiniMind 的 MoE 实现核心代码（简化版）：

```
class MoEGate(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.top_k = config.num_experts_per_tok  # 1
        self.n_routed_experts = config.n_routed_experts  # 4
        self.scoring_func = 'softmax'
        self.gate = nn.Linear(config.dim, self.n_routed_experts, bias=False)

    def forward(self, hidden_states):
        logits = self.gate(hidden_states)  # [B*T, num_experts]
        scores = logits.softmax(dim=-1)
        topk_weight, topk_idx = torch.topk(scores, k=self.top_k, dim=-1)
        topk_weight = topk_weight / (topk_weight.sum(dim=-1, keepdim=True) + 1e-20)
        return topk_idx, topk_weight, scores
```

5.2 专家分发与计算

```
class MOEFeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.gate = MoEGate(config)
        self.experts = nn.ModuleList([
            FeedForward(config) for _ in range(config.n_routed_experts)
        ])

    def forward(self, x):
        identity = x
        orig_shape = x.shape
        x = x.view(-1, x.shape[-1])  # [B*T, D]
```

```
        topk_idx, topk_weight, scores = self.gate(x)

        # 按专家分组处理
        y = torch.zeros_like(x)
        for i, expert in enumerate(self.experts):
            # 找到选择了第 i 个专家的所有 token
            mask = (topk_idx == i).any(dim=-1)
            if mask.any():
                token_input = x[mask]
                expert_out = expert(token_input)
                # 提取对应权重并加权
                idx_mask = (topk_idx[mask] == i)
                weight = topk_weight[mask][idx_mask].unsqueeze(-1)
                y[mask] += weight * expert_out

        return y.view(*orig_shape)
```

5.3 负载均衡 Loss 的集成

在训练循环中，负载均衡 loss 会被加入总损失：

```
# 训练步骤
outputs = model(input_ids, labels=labels)
lm_loss = outputs.loss

# 收集所有 MoE 层的负载均衡 loss
balance_loss = 0
for layer in model.layers:
    if hasattr(layer.feed_forward, 'gate'):
        balance_loss += layer.feed_forward.gate.balance_loss

total_loss = lm_loss + alpha * balance_loss
total_loss.backward()
```

六、MoE 的优势与挑战

6.1 优势

1. **扩展性强**：可以通过增加专家数量来扩大模型容量，而不线性增加计算量
2. **参数效率**：在相同计算预算下，MoE 能存储更多"知识"
3. **专业化**：不同专家可能自然地学会处理不同类型的输入（虽然实际中专业化程度有限）

6.2 挑战

1. **训练速度**：原生 PyTorch 中，逐个专家计算 + 条件分发的 overhead 很大
2. **显存占用**：虽然激活参数少，但所有专家的参数都要存在 GPU 上

3. **负载均衡**：需要精心设计 loss 来防止专家塌缩
4. **通信开销**：在分布式训练中，不同专家可能分布在不同 GPU 上，token 路由需要跨设备通信（Expert Parallelism）
5. **推理复杂度**：需要先过 Router 再分发，比 Dense 模型复杂

6.3 工程优化方向

- **Triton Kernel**：用自定义 GPU kernel 实现高效的 token 分发
- **Expert Parallelism**：将不同专家放在不同 GPU 上
- **Capacity Factor**：限制每个专家处理的 token 数量上限
- **Megablocks**：Facebook 提出的高效 MoE 实现库

🎤 面试考点

Q1: MoE 的基本原理是什么?

答：MoE（Mixture of Experts）用多个并行的 FFN 专家替代标准 Transformer 中的单个 FFN。通过 Router（路由器）决定每个 token 送给哪些专家处理。这样可以在增加模型参数容量的同时控制计算量——每个 token 只激活部分专家，实现稀疏激活。

Q2: 什么是 Top-K Routing？MiniMind 用的是 Top 几?

答：Top-K Routing 指 Router 为每个 token 选择分数最高的 K 个专家来处理。MiniMind-3-MoE 使用 Top-1 Routing，即每个 token 只送给 1 个专家。Top-1 计算量最小但表达能力有限；Top-2 更常见，是计算量和效果的折中。

Q3: 什么是专家塌缩（Expert Collapse）？怎么解决?

答：专家塌缩是指训练过程中所有 token 都倾向于被路由到同一个专家，其他专家得不到训练的现象。原因是正反馈循环。解决方法是添加负载均衡 Loss：$\mathcal{L}_{balance} = \alpha \cdot N \cdot \sum f_i \cdot P_i$，鼓励 Router 把 token 更均匀地分配给各个专家。

Q4: MoE 的"总参数"和"激活参数"分别是什么?

答：总参数 = 模型中所有参数的数量（包括所有专家）；激活参数 = 处理单个 token 时实际参与计算的参数数量。以 MiniMind-3-MoE 为例，总参数 198M，激活参数 64M，因为 Top-1 Routing 每次只激活 4 个专家中的 1 个。

Q5: MoE 中 Router 是怎么实现的?

答：Router 通常就是一个线性层（`nn.Linear(hidden_dim, num_experts)`）加 Softmax。输入 token 的 hidden state，输出对每个专家的概率分布。非常简单但很关键——Router 的训练效果直接决定了 token 分配的合理性。

Q6: MoE 相比 Dense 模型有什么优劣?

答： 优势：(1) 相同计算预算下参数容量更大；(2) 扩展性好，增加专家不线性增加计算量。 劣势：(1) 原生实现训练速度较慢（kernel 调度开销）；(2) 显存占用更大（要存所有专家参数）；(3) 需要处理负载均衡问题；(4) 分布式训练的通信开销。

Q7: 在实际训练中，MoE 的专家真的会"专业化"吗?

答：在一定程度上会。研究发现不同专家可能对不同语言、不同领域或不同 token 类型有偏好，但这种专业化通常不是非常明显和清晰的。专业化的程度受到负载均衡 loss 的影响——负载均衡 loss 越强，越倾向于均匀分配，专业化程度越低。

✅ 自测题

1. **填空**：MiniMind-3-MoE 有 _ ***个专家，使用 Top-***_ Routing，总参数 _ ***M，激活参数*** _M。
2. **判断**：MoE 模型中 Attention 层也被替换成了多个专家。（对/错）
3. **选择**：负载均衡 Loss 的目的是什么?
4. A. 提高模型精度
5. B. 防止专家塌缩
6. C. 加速训练
7. D. 减少参数量
8. **简答**：为什么 MoE 在原生 PyTorch 中训练比 Dense 模型慢?
9. **计算**：一个 MoE 模型有 8 个专家，每个专家参数量 500M，Attention 等非 FFN 部分参数量 2B，使用 Top-2 Routing。请计算总参数量和激活参数量。

▼ 参考答案

1. 4 个专家，Top-1 Routing，198M 总参数，64M 激活参数 2. 错。MoE 只替换 FFN 层，Attention 层保持不变。 3. B。防止所有 token 涌向同一个专家。 4. 因为 MoE 需要根据 Router 的输出动态分发 token 给不同专家，这涉及条件执行和 tensor 的动态索引操作。在原生 PyTorch 中，这些操作无法被 GPU 高效并行化（需要逐个专家计算），导致 GPU kernel 调度开销大。 5. 总参数 = 2B + 8 × 500M = 6B；激活参数 = 2B + 2 × 500M = 3B（Top-2 激活 2 个专家）

下一节预告

下一节我们将学习 **推理优化**，包括 KV-Cache、YaRN 长度外推、生成策略（Top-K/Top-P/Temperature）等关键技术，让模型又快又好地生成文本。